

Technical Design Document

ONE DROP



Abdelghafour Moumad – Game Programmer

Ahmed Zine – Game Artist

Mohamed Chadi Gmira – Game Designer

Tableau des versions

N°Version	Date	Auteur(s)	Description
1.0	15/02/2026	Abdelghafour Moumad	Version initiale

Table des matières

Tableau des versions	1
1 Présentation du jeu	4
1.1 Gameplay principal	4
1.2 Périmètre technique	4
2 Plateformes et spécifications hardware	4
2.1 Plateforme cible	4
2.2 Configuration minimale	4
2.3 Configuration recommandée	4
3 Environnement de développement	5
3.1 Moteur de jeu	5
3.2 IDE et outils de développement	5
3.3 Bibliothèques et packages	5
4 Risques techniques	5
5 Pipeline de production	5
5.1 Assets graphiques	5
5.2 Assets audio	6
5.3 Shaders / FX	6
6 Architecture logicielle	6
6.1 Architecture runtime actuelle	6
6.2 Responsabilités	7
7 Mise en œuvre technique des mécanismes de gameplay	7
7.1 Mouvement horizontal	7
7.2 Détection sol/mur	7
7.3 Slide dash (double tap)	7
7.4 Wall climb + wall jump	8
7.5 Déformation visuelle soft-body	8
8 Techniques de rendu graphique & complexité	8
8.1 Pipeline de rendu	8
8.2 Post-processing	8
8.3 Complexité de scène (objectif)	8
9 Performances	8
9.1 Optimisations déjà en place	8
10 Menus & HUD	9
11 Audio	9

12 Convention d'écriture de code	9
12.1 Nommage	9
12.2 Organisation des scripts	9
12.3 Règles de code C# / Unity	9
12.4 Commentaires et documentation	10
12.5 Format obligatoire des en-têtes de scripts	10
12.6 Format obligatoire des commentaires de modification	10
12.7 Logs, debug et erreurs	10
12.8 Qualité et revue de code	10
12.9 Règles de collaboration (plusieurs programmeurs)	10
13 Équipe et organisation	11
13.1 Méthodologie	11
13.2 Répartition des responsabilités	11
13.3 Rituels d'équipe	11
13.4 Workflow de production (Git)	11
13.5 Definition of Done (DoD)	11
13.6 Gestion de la communication	12
13.7 Gestion des risques organisationnels	12
13.8 Composition de l'équipe	12

1 Présentation du jeu

ONE DROP est un jeu de plateforme 2D orienté physique. Le joueur contrôle une goutte d'eau dans un environnement volcanique, avec un focus sur la précision du mouvement et le feedback visuel organique.

1.1 Gameplay principal

- Mouvement horizontal avec accélération contrôlée
- Saut au sol + wall jump
- Wall climbing
- Slide dash via double tap gauche/droite
- Tir de projectile d'eau (normal et chargé)
- Adaptation de rotation selon la pente du sol
- Déformation visuelle soft-body (squash & stretch)

1.2 Périmètre technique

- Contrôleur principal : `OneDropController2D`
- Détections sol/mur par `Physics2D.BoxCast`
- Projectile : `WaterDropProjectile2D`
- Caméra follow custom : `CameraFollow2D`
- Effets visuels de niveau lave : `LavaPlatformVisual2D`, `LavaBackgroundFX2D`
- Variante soft-body physique complète (prototype) : `WaterBlobCharacter2D`

2 Plateformes et spécifications hardware

2.1 Plateforme cible

Le jeu est développé pour **PC (Windows 10/11)**.

2.2 Configuration minimale

Configuration minimale

- **CPU** : Intel i5 (4^e gen) ou équivalent
- **RAM** : 8 GB
- **GPU** : Intel UHD / GTX 750 ou équivalent
- **Performance cible** : 60 FPS en 1080p

2.3 Configuration recommandée

Configuration recommandée

- **CPU** : Intel i7 / Ryzen 5+
- **RAM** : 16 GB
- **GPU** : GTX 1060 ou supérieur

3 Environnement de développement

3.1 Moteur de jeu

Unity 6 – 6000.1.7f1 avec Universal Render Pipeline 17.3.0.

3.2 IDE et outils de développement

- **IDE** : Visual Studio 2022
- **Versionning** : Git + GitHub
- **Gestion tâches/doc** : Miro

3.3 Librairies et packages

Package	Utilisation
com.unity.inputsystem (1.17.0)	Gestion input clavier + gamepad
com.unity.render-pipelines.universal (17.3.0)	Rendu 2D URP
com.unity.ugui (2.0.0)	Base UI (menus/HUD futurs)

4 Risques techniques

Risque	Impact	Solution
Instabilité de mouvement sur angles de collision	Blocage latéral/ressenti imprécis	Material 2D friction=0 + filtre par normales de contact (anti edge-stick)
Détections mur/sol incohérentes	Wall jump/climb non fiables	BoxCast dimensionné depuis collider + LayerMask dédiés
Surcoût visuel soft-body	Baisse FPS sur scènes chargées	Lerp borné, logique visuelle séparée de la physique gameplay
Dualité architecture (One-Drop vs WaterBlob physique)	Dette technique et confusion d'intégration	Garder OneDrop-Controller2D comme baseline jouable, WaterBlob-Character2D en branche R&D

5 Pipeline de production

5.1 Assets graphiques

- **Format** : PNG sprites 2D
- **Max size** : 2048x2048
- **Import Unity** : Sprite (2D and UI), compression adaptée plateforme

5.2 Assets audio

- **État actuel** : pipeline audio global non implémenté dans les scripts runtime
- **Prévision** : WAV pour SFX courts, OGG pour boucles musicales

5.3 Shaders / FX

- Matériaux runtime pour blob et plateformes lave
- Pulsation emissive et variations colorimétriques dynamiques

6 Architecture logicielle

6.1 Architecture runtime actuelle

Player (WaterBlobPlayer)

- OneDropController2D
- Rigidbody2D + BoxCollider2D
- (Optionnel) WaterBlobCharacter2D + WaterBlobMeshRenderer2D
- (Optionnel) WaterBlobGooglyEyes2D

World

- LavaPlatformVisual2D
- LavaBackgroundFX2D
- WaterDropProjectile2D (spawn runtime)

Camera

- CameraFollow2D

6.2 Responsabilités

- **OneDropController2D** : mouvement principal, saut, climb, wall jump, slide, tir, rotation, déformation visuelle.
- **WaterDropProjectile2D** : déplacement projectile, collision trigger, impulsion à l'impact.
- **CameraFollow2D** : suivi caméra lissé avec limites optionnelles.
- **LavaBackgroundFX2D** : parallax et réactions visuelles à la vitesse du joueur.
- **LavaPlatformVisual2D** : génération déco runtime et pulsation emissive.

7 Mise en œuvre technique des mécanismes de game-play

7.1 Mouvement horizontal

Listing 1 – Mouvement horizontal (OneDropController2D)

```
1 float targetX = inputX * moveSpeed;
2 float currentX = rb.linearVelocity.x;
3 float newX = Mathf.MoveTowards(currentX, targetX, acceleration *
   Time.fixedDeltaTime);
4 rb.linearVelocity = new Vector2(newX, rb.linearVelocity.y);
```

7.2 Détection sol/mur

Listing 2 – Détection via BoxCast

```
1 Vector2 size = GetCastSize();
2 RaycastHit2D groundHit = Physics2D.BoxCast(
3     transform.position,
4     size,
5     0f,
6     Vector2.down,
7     groundCheckDistance,
8     groundMask
9 );
10
11 RaycastHit2D wallHit = Physics2D.BoxCast(
12     transform.position,
13     size,
14     0f,
15     Vector2.right,
16     wallCheckDistance,
17     wallMask
18 );
```

7.3 Slide dash (double tap)

- Détection double appui gauche/droite dans une fenêtre temporelle (doubleTapWindow)

- Vitesse imposée temporairement (`slideSpeed`)
- Cooldown de sécurité (`slideCooldown`)

7.4 Wall climb + wall jump

- Gravité réduite/annulée en état climbing
- Impulsion opposée au mur pour wall jump
- Timer anti-spam sur les sauts (`minJumpInterval`)

7.5 Déformation visuelle soft-body

- Squash/stretch piloté par vitesse, accélération, impact d'atterrissage, pente et état mur
- Interpolation continue pour éviter les pops visuels
- Physique de collision conservée stable via rigidbody + collider

8 Techniques de rendu graphique & complexité

8.1 Pipeline de rendu

URP 2D avec matériaux runtime (sprites, mesh blob, plateformes lave).

8.2 Post-processing

- Non centralisé dans les scripts actuels
- Possibilité d'ajout dans Volume Profile URP selon budget performance

8.3 Complexité de scène (objectif)

- Draw calls maîtrisés par usage simple de matériaux
- Limiter particules et géométrie décorative runtime sur plateformes nombreuses

9 Performances

Objectif

- 60 FPS stable
- 1080p
- Frame time cible : < 16.67 ms

9.1 Optimisations déjà en place

- Détections physiques bornées (distance/masques)
- Material physique runtime friction 0 pour éviter collisions parasites
- Interpolations temporelles contrôlées pour les déformations visuelles

10 Menus & HUD

- **État actuel** : pas de système menu/HUD complet dans les scripts fournis
- **Plan** : menu principal + pause + HUD minimal (retour visuel sur charge tir et état joueur)

11 Audio

- **État actuel** : pas de `AudioManager` global implémenté dans ce build
- **Plan** : manager centralisé SFX/Music + pool d'`AudioSource`

12 Convention d'écriture de code

12.1 Nommage

Type	Convention	Exemple
Classes	PascalCase	<code>OneDropController2D</code>
Interfaces	Préfixe I + PascalCase	<code>IInteractable</code>
Méthodes publiques	PascalCase (verbe d'action)	<code>TryShootWater()</code>
Méthodes privées	PascalCase	<code>ApplyHorizontalMovement()</code>
Champs sérialisés privés	camelCase + <code>[SerializeField]</code>	<code>moveSpeed</code>
Champs privés non sérialisés	<code>_camelCase</code>	<code>_shotCooldownTimer</code>
Constantes	UPPER_CASE	<code>MAX_SPEED</code>
Booléens	Préfixe <code>is/has/can</code>	<code>isClimbing</code>
Layers/Tags (Unity)	Noms explicites et stables	<code>Ground, Wall</code>

12.2 Organisation des scripts

- Un script = une responsabilité principale (principe SRP).
- Les scripts gameplay sont regroupés sous `Assets/Scripts/WaterBlob`.
- Ordre interne recommandé d'une classe Unity : `Serialized Fields` → `Private Fields` → `Unity Callbacks` (`Awake/Start/Update/FixedUpdate`) → `Public API` → `Private Methods`.
- Les dépendances obligatoires sont déclarées avec `[RequireComponent(...)]`.
- Les valeurs de gameplay doivent être paramétrables dans l'Inspector via `[SerializeField]`.

12.3 Règles de code C# / Unity

- Utiliser `FixedUpdate()` pour les calculs physiques (`Rigidbody2D`, forces, vitesses).
- Utiliser `Update()` pour la lecture input et les timers non physiques.
- Éviter `Find()` en boucle ; faire les résolutions de références dans `Awake/Start` (ou avec retry contrôlé).
- Préférer `Try...` quand une action peut échouer (ex : `TryShootWater()`).
- Éviter toute allocation évitable dans la boucle de frame.
- Ne pas "hardcoder" les masks de collision ; exposer des `LayerMask`.

12.4 Commentaires et documentation

- Les méthodes publiques sont documentées avec XML Summary.
- Les commentaires inline sont réservés aux choix non évidents (bugs évités, contraintes moteur, trade-offs).
- Un commentaire doit expliquer le **pourquoi**, pas décrire une ligne évidente.
- Toute modification significative doit contenir un commentaire daté avec initiales.

12.5 Format obligatoire des en-têtes de scripts

Template d'en-tête C#

```
// File: OneDropController2D.cs
// Author: AM
// Created: 2026-02-15
// Last Update: 2026-02-15 (AM) - Added charged shot cooldown fix
// Purpose: Player movement, climb, wall jump, slide and water
shooting.
```

12.6 Format obligatoire des commentaires de modification

- Format imposé : `// [JJ/MM/AAAA] [Initiales] Description courte.`
- Exemple : `// [15/02/2026] [AM] Prevent projectile self-collision at spawn.`
- Les tags `TODO`, `FIXME`, `NOTE` doivent toujours inclure initiales + date.

12.7 Logs, debug et erreurs

- Format de log recommandé : `[SystemName] Message (ex : [OneDrop] Projectile prefab missing).`
- `Debug.LogWarning()` pour les cas récupérables, `Debug.LogError()` pour les états invalides.
- Les outils visuels de debug (Gizmos) doivent être activables/désactivables.

12.8 Qualité et revue de code

- Toute feature gameplay doit être testée en mode Play avant merge.
- Toute PR doit inclure : objectif, fichiers modifiés, risques, et plan de test.
- Pas de régression de contrôle joueur sans validation explicite de l'équipe.

12.9 Règles de collaboration (plusieurs programmeurs)

- Chaque ticket est assigné à un seul responsable technique (owner).
- Aucune fusion directe sur la branche principale : passage obligatoire par Pull Request.
- Revue minimale : 1 approbation d'un autre programmeur avant merge.
- Les conflits de conventions sont arbitrés par ce document (source de vérité).
- Toute nouvelle convention ajoutée doit être datée et validée en équipe.

13 Équipe et organisation

13.1 Méthodologie

Scrum simplifié (sprints de 2 semaines) avec planification courte et validation continue.

- Durée d'un sprint : 10 jours ouvrés.
- Priorisation : backlog ordonné par criticité gameplay (contrôles, lisibilité, stabilité, performance).
- Objectif de sprint : livrer un incrément jouable et testable.
- Démonstration de fin de sprint : build interne + liste des points validés/non validés.

13.2 Répartition des responsabilités

Rôle	Responsabilités principales	Livrables attendus
Game Programmer	Gameplay 3C, physique 2D, tir, intégration technique, correctifs bugs	Scripts C#, scènes jouables, notes techniques, PR testées
Game Artist	Direction visuelle, sprites, cohérence esthétique des environnements lave/eau, intégration visuelle	Assets 2D (sprites/textures), variantes d'assets, previews visuelles
Game Designer	Boucle de jeu, difficulté, level flow, équilibrage des mécaniques	GDD mis à jour, specs de niveaux, tableaux d'équilibrage

13.3 Rituels d'équipe

- **Daily rapide (10-15 min)** : fait / à faire / blocage.
- **Sprint Planning** : découpage des tâches, estimation, définition de l'objectif sprint.
- **Sprint Review** : validation des fonctionnalités terminées sur build.
- **Rétrospective** : actions d'amélioration process pour le sprint suivant.

13.4 Workflow de production (Git)

- Branche principale protégée : `main`.
- Une fonctionnalité = une branche : `feature/nom-court`.
- Nommage des correctifs : `fix/nom-court`.
- Intégration uniquement via Pull Request avec revue.
- Message de commit recommandé : `type(scope): description (ex: feat(player): add charged water shot)`.

13.5 Definition of Done (DoD)

- La fonctionnalité fonctionne en Play Mode sans erreur console bloquante.
- Le comportement est validé par au moins un autre membre.

- Le code respecte les conventions d'écriture définies dans ce document.
- Les paramètres gameplay sont exposés dans l'Inspector quand pertinent.
- La PR contient un plan de test et les risques identifiés.

13.6 Gestion de la communication

- Canal synchrone : Discord (questions rapides, blocages).
- Canal asynchrone : GitHub (issues/PR) + Notion/Trello (suivi backlog).
- Les décisions techniques importantes sont tracées par écrit (issue ou page Notion).

13.7 Gestion des risques organisationnels

- **Risque** : retard asset ou feature critique. **Mitigation** : version fallback simple mais jouable.
- **Risque** : divergence vision game design / implémentation. **Mitigation** : validations intermédiaires hebdomadaires.
- **Risque** : dette technique en fin de projet. **Mitigation** : créneau récurrent de refactor/correctifs à chaque sprint.

13.8 Composition de l'équipe

Rôle	Membre	Responsabilité dominante
Game Programmer	Abdelghafour Moumad	Gameplay systems, architecture technique, intégration Unity
Game Artist	Ahmed Zine	Assets visuels, lisibilité scène, direction artistique
Game Designer	Mohamed Chadi Gmira	Boucle de jeu, équilibrage, progression des niveaux